

Drive Scribus from Claude.

scribus-mcp is a Model Context Protocol server that wraps the Scribus 1.7 Scriptor API. Claude uses it to compose, style, and export desktop-publishing documents — through a live Scribus window or a one-shot headless invocation.

MCP TOOLS

125

across 17 modules

TESTS PASSING

136

Phase 1 + 2 green

BACKENDS

2

headless + interactive

EXTERNAL SERVICES

0

stdlib + scribus only

Composable

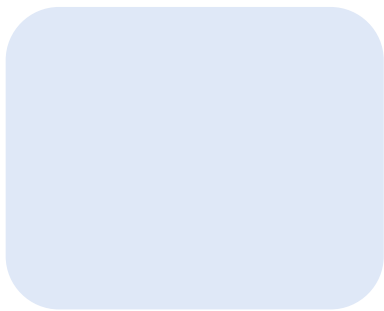
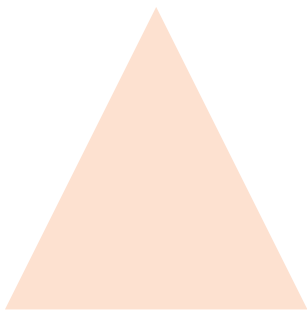
All MCP primitives — Tools, Resources, Prompts — are first-class. Claude discovers them, calls them, and gets typed JSON back.

Live or batched

Two interchangeable backends: persistent bridge inside a running Scribus, or a fresh headless spawn per call. Auto-selected.

Pattern-rich

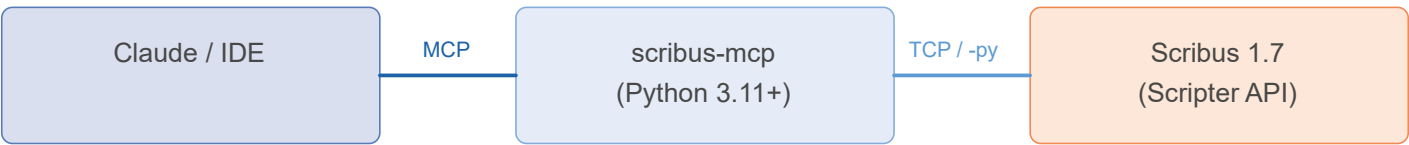
High-level tools cover charts, callouts, KPIs, tables, QR codes, syntax-highlighted code blocks, layouts, and Markdown import — one call replaces dozens of primitives.



Read on for the architecture, the tool surface, and how to install it.

Architecture

One Python server speaks MCP to the client and Scriber to Scribus. Two backends sit behind a single tool surface — chosen per call, with sensible defaults.



TWO BACKENDS, ONE TOOL SURFACE

Headless

Spawn ``scribus -g -ns -py /tmp/job.py`` per call. Stateless, ~3 s per spawn. Best for batch jobs and CI: generate manuals, briefs, and PDFs unattended.

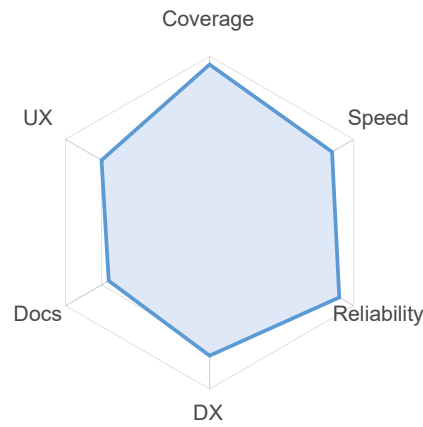
Interactive bridge

A small `.spy` script inside a running Scribus listens on TCP loopback. The MCP dispatches calls onto Scribus's Qt main thread via QTimer. ~50 ms per call. Best for live editing assistance — Claude composes while you watch.

BACKEND COMPARISON

Aspect	Headless	Interactive bridge
Latency / call	~3 s	~50 ms
Persistence	stateless	doc state preserved
GUI	none	responsive (with Qt binding)
Best for	CI batch	live editing
Discovery	spawn	JSON discovery file + token
Failure mode	process exits	tool returns ok=false

WHERE WE ARE



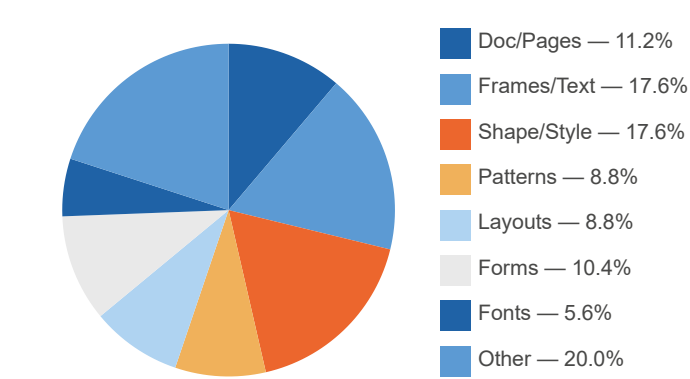
Coverage and reliability come from 85 integration tests against a live Scribus, ABI-matched PyQt6 install for interactive mode, and per-tool error paths that surface as `{ok: false, error: ...}` for the model to self-correct.

Documentation and UX are still maturing: the patterns library will keep growing.

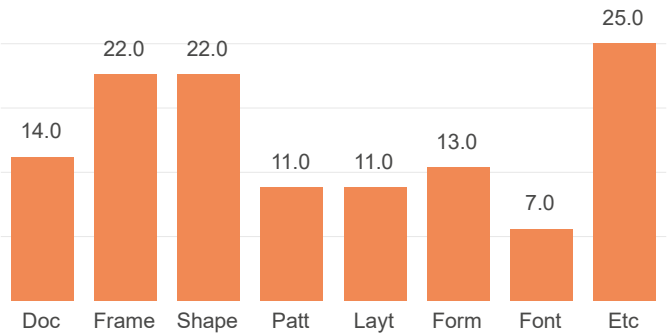
What you can call

Tools are grouped by domain. Each one returns a structured dict so Claude can reason about success, failure, and any object names it created.

TOOL DISTRIBUTION



TOOLS PER CATEGORY



HIGH-LEVEL PATTERNS — ONE CALL EACH

create_code_sample

Pygments syntax-highlighted code block with monospace font + per-token color.

create_radar_chart

N-axis polar chart with concentric rings, axis labels, and a data overlay polygon.

create_bar_chart

Vertical bars with gridlines, optional value labels above each bar.

create_pie_chart

Polygon-approximated slices with legend + clean outer ring.

create_timeline

Horizontal axis + tick marks + label-above + optional date-below.

create_callout_box

Bordered, lightly shaded box with title and justified body.

create_kpi_row

Auto-laid row of KPI tiles — pass items, get an evenly-spaced row back.

create_card_grid

2-D grid of accent-bordered cards with eyebrow / title / body.

create_qr_code_block

QR / EAN13 / Code128 / Data Matrix via Scribus's BWIPP, optional caption.

Where this fits

Anywhere you generate or maintain print-quality documents — and want an LLM to do it without inventing geometry or writing PostScript by hand.

BRAIN20 / HARDWARE · ACCENT: LEFT

Product manuals

Convert a Markdown spec + a parts CSV into a styled, indexed user manual. Headless mode runs unattended in CI; PDF/X-4 output is print-ready.

INTERNAL ONE-PAGERS · ACCENT: TOP

Brief documents

Database schemas, deployment briefs, postmortems. The MCP composes headers, KPI tiles, comparison tables, and code blocks from a structured prompt.

PERIODIC DELIVERABLES · ACCENT: RIGHT

Dashboards & reports

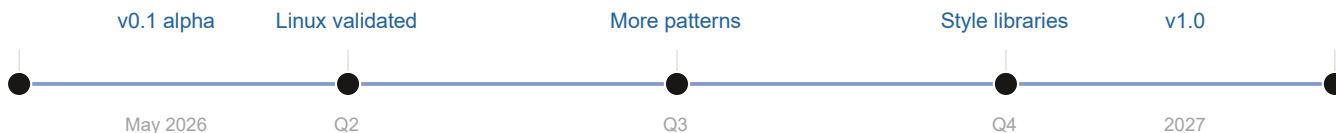
Weekly KPI snapshots, sales reports, status updates. Pie / bar / radar / timeline patterns — one tool call per chart, parameterised by your data.

DATASHEETS, FLYERS · ACCENT: BOTTOM

Marketing collateral

Templated cards, pitch decks rendered as PDFs, datasheets with auto-generated QR codes that link to per-product pages. Same MCP, different prompt.

WHERE THIS PROJECT IS HEADED



MIT-licensed and source-available. Patterns add as much value as primitives — if your team has a recurring layout, contributing it as a single tool benefits every other user. The repository link is on the next page.

Getting started

Three steps from zero to a Scribus driven by Claude.

1

Install Scribus + the MCP

Scribus 1.7.x from scribus.net; the MCP via `pip install -e .` in a Python 3.11+ venv. Verify with `scribus-mcp --help`.

2

(Optional) Enable the live bridge

Linux: `apt install python3-pyqt6` (system Python). Windows: run `scripts/install-pyqt-windows.ps1` as administrator — auto-detects Scribus's Qt minor version and pins a matching PyQt6.

3

Wire it into Claude Code

Add `scribus-mcp` to your MCP server config. Open Scribus + the bridge, or let Claude call tools with `mode="headless"` (no Scribus needed).

HELLO WORLD — MINIMAL EXAMPLE

claude-mcp-config.json

```
1 | {  
2 |     "mcpServers": {  
3 |         "scribus": {  
4 |             "command": "scribus-mcp",  
5 |             "env": { "SCRIBUS_BIN": "/usr/bin/scribus" }  
6 |         }  
7 |     }  
8 | }
```

REPOSITORY



caewa/scribus-mcp

Generated by scribus-mcp itself — every page on this PDF.